Repository: https://github.com/sunitazenith07-lang/EventHive-Open-Source-Evaluation

OrderOps — Zomato-Inspired Order Failure, Cancellation & Refund Reconciliation

Track: Food-delivery operations | **Language:** Python 3.10+ | **Time:** 24-hour evaluation

Scenario

A food-delivery platform can confirm payment before the restaurant accepts an order. The restaurant may reject it; customers may cancel before dispatch; and refund notifications can arrive late or more than once. The service must not duplicate a refund or confuse a partial refund with a completed refund.

This is a simulated, offline project inspired by food-delivery industry workflows, including challenges faced by services such as Zomato. It is not Zomato's internal software, is not affiliated with Zomato, and does not use real customers or payment APIs.

Required behaviour

1. Process the supplied events: payment_confirmed, restaurant_accepted, restaurant_rejected, cancel_requested, dispatch_started, order_delivered and refund_processed.

2. Enforce valid transitions. A cancellation after dispatch must not silently change the order to CANCELLED.

3. Use event_id for idempotency. Replaying an event must not change order or refund state a second time.

4. Use integer paise. Refund amounts must be positive, cannot exceed the outstanding refundable amount, and apply only to eligible cancelled/rejected orders that were paid.

5. Track refund_due_paise and refunded_paise separately. Partial refunds remain partial; set payment status to REFUNDED only when the full due amount has been processed.

6. Reconcile internal and provider summaries by order ID. Report missing records, amount mismatches and status mismatches separately and deterministically.

7. Handle empty input, missing fields, unknown IDs, invalid event types and duplicates without corrupting state.

Setup

Run: `python -m pip install -r requirements-dev.txt`, then `python -m pytest -q`, then `python -m main`.

Repository reading checklist and dataset

Before editing, read `README.md` and `STUDENT_START_HERE.md`, then inspect the whole repository—not only this PDF. Read `CONTRIBUTING.md`, `.github/PULL_REQUEST_TEMPLATE.md`, the test/template documents, `service.py`, `models.py`, `fixtures.py`, `main.py`, every file under `tests/`, and the workflow files under `.github/workflows/`.

The expanded synthetic dataset is in `data/records.json`: **150 internal orders, 163 provider records, and 219 event examples**, plus a reconciliation edge-case catalog. Read `data/README.txt` for its schema and intentional anomalies. It covers cancellation before and after dispatch, restaurant rejection, duplicate or malformed events, refund bounds, provider-only records, and amount/status mismatches. This is simulated industry-inspired data—not Zomato's internal data. The offline demo uses `fixtures.py`; the larger JSON is additional investigation and test material.

Run the baseline tests before editing and record their actual result. Do not delete or weaken tests to make CI green.

Work to submit

- Read the source and run the starter tests before editing.
- Treat the requirements above as the acceptance contract. Investigate failing tests, trace root causes, and correct the underlying behaviour; do not hard-code fixture output.
- Add regression tests for fixes and document at least five edge cases in `TESTING.md`.
- Submit one pull request against the original repository with code, tests, a root-cause summary and known limitations. Do not merge it.
- Include `ORDER_FLOW_NOTES.md` with one example event sequence, expected states and three edge cases.

Mandatory problem-solving documentation — all three files required

Submit these three completed Markdown files at the repository root in your pull request. They are evaluated; code alone is not a complete submission.

- `approach.md` — record your starting test result, the code/data flow you explored, key hypotheses, experiments you ran, implementation strategy, and why you chose it.
- `challenges.md` — **COMPULSORY**. Describe concrete obstacles or confusing behaviours, what you tried, evidence from tests/logs, how you resolved them, and any unresolved limitations. If you did not get stuck, state that honestly and describe the hardest edge case you deliberately tested; do not invent a struggle.
- `optimizations.md` — describe optimizations considered or implemented, why you chose or rejected them, correctness/readability/performance trade-offs, and future improvements. Separate measured improvements from ideas you did not measure.

Keep the notes concise, specific and evidence-based. Describe your actual work, tests and technical decisions; do not provide private hidden chain-of-thought or a stream-of-consciousness transcript. Generic or copied text does not demonstrate your process. Each file is worth 5 marks. A missing or empty file receives 0/5 for that component and makes the documentation requirement incomplete.

Optional stretch work

Choose at most one after the core behaviours work: export an awaiting-refund CSV report; flag out-of-order events; or summarise orders by state.

Testing and GitHub Actions

Some visible tests are expected to fail on the untouched starter because key behaviours are incomplete. The exact initial test count is not fixed. GitHub Actions runs against push and pull-request changes and publishes visible test counts. A red check before your fixes is expected. Do not remove, skip, weaken or bypass tests, or edit CI to evade failures.

Individual-work policy

AI chatbots, AI coding assistants, generated patches/tests and asking another person to write/debug the solution are prohibited. Ordinary non-AI documentation is allowed. Do not upload challenge code/data to external services. You must be able to explain your code and make a small live change if asked. A code walkthrough can support integrity checks, but no automated detector proves whether AI was used.

Rubric — 100 marks

- Codebase understanding and order-state flow: 10
- Root-cause analysis and correctness: 20
- State transitions, idempotency and refund safety: 25
- Regression tests and edge cases: 15
- Code quality and maintainability: 5
- Required problem-solving documentation: 15 (approach.md: 5, challenges.md: 5, optimizations.md: 5)
- Pull request, order-flow notes and live explanation: 10

The three process documents are assessed for specific, honest, evidence-based technical reflection, not writing style.